

Veritabanı 2 Dönem Projesi - Mülakat ve Sunum Çalışma Rehberi

Bu rehber, projenizi hocanıza sunarken gelebilecek teknik sorulara karşı hazırlanmış "hap bilgi" niteliğinde bir soru-cevap dosyasıdır.

Soru 1: Örnek projen genel olarak ne yapıyor? Amacı nedir?

Cevap: Bu proje, 15 tablodan oluşan tam teşekküllü bir **Hastane Otomasyon Sistemi**dir. Hasta, Doktor ve Sekreter olmak üzere üç farklı kullanıcı rolü vardır. Hastalar randevu alabilir; sekreterler doktor, branş ve randevu yönetimini yapar; doktorlar ise kendilerine atanan randevuları görebilir. Projenin asıl amacı sadece arayüz yapmak değil; arka planda **Trigger, View, Transaction ve Procedure** gibi ileri seviye SQL mimarilerini kullanarak verinin güvenliğini ve bütünlüğünü doğrudan veritabanı katmanında sağlamaktır.

Soru 2: Logları (kayıtları) nerede tutuyorsun? C# içinde mi, veritabanında mı?

Cevap: Logları tamamen **veritabanı içinde fiziksel tablolarda** (`Table_kullanici_log` ve `Table_randevu_log`) tutuyorum. Buradaki en önemli nokta şudur: Bu loglama işlemini C# (uygulama) tarafında kod yazarak yapmıyorum. Loglar, SQL Server içindeki **Trigger (Tetikleyici)** yapısıyla oluşturuluyor. Uygulama içerisine yeni eklediğim "Sistem Logları" paneli, sadece bu tablolarda biriken verileri ekrana (okumak/SELECT için) yansıtır.

Soru 3: Stored Procedure (Saklı Yordam) kullandın mı? Neden kullandın?

Cevap: Evet, projede toplam **13 adet Stored Procedure** (örneğin `sp_RandevuOlustur`, `sp_HastaGuncelle`, `sp_DoktorKaydet` vb.) kullandım. Bütün CRUD (Ekleme, Silme, Güncelleme) işlemlerini bunlar üzerinden yapıyorum. *Neden kullandım?* 1. Performans: SQL Server bu sorguları bir kez derleyip (compile) önbelleğe aldığı için daha hızlı çalışır. 2. Güvenlik: Uygulama üzerinden SQL Injection saldırılarını engeller. 3. Temiz Kod: C# içerisine uzun SQL metinleri yazmak yerine sadece yordamın adını çağırmak kod karmaşasını önler.

Soru 4: Transaction kullandın mı? Hangi işlemde ve neden?

Cevap: Evet, projede **2 adet Transaction** kullandım. Bunlardan en önemlisi **Randevu Alma** (`sp_HastaRandevuAl`) işlemindedir. Hasta bir slota tıklayıp randevu aldığı anda arka planda iki şey olur: 1. `Table_randevu` tablosu güncellenir. 2. `Table_odeme` tablosuna otomatik bir muayene ücreti kaydı atılır. Eğer ödeme kaydı atılırken veritabanında bir hata çıkarsa, randevu da alınmamış sayılsın ve veritabanı bozulmasın diye **ROLLBACK** işlemi yaptırıyorum. Hata olmazsa **COMMIT** ile işlemi kalıcı hale getiriyorum (ACID prensipleri).

Soru 5: View kullandın mı? View kullanmanın tablodan farkı nedir?

Cevap: Evet, toplam **5 adet View** (örneğin `vw_YonetimOzeti`, `vw_DoktorBransListesi`) kullandım. Tablodan farkı şudur: View'lar fiziksel olarak diskte yer kaplayan veriler tutmazlar, onlar "sanal" tablolardır. Projemdeki `vw_DoktorBransListesi` view'ı; doktor, poliklinik, branş ve oda tablolarını `JOIN` ile arka planda birleştirir. Ben C# tarafında upuzun bir `JOIN` yazmak yerine sadece `SELECT * FROM vw_DoktorBransListesi` diyerek karmaşık veriyi saniyeler içinde çekiyorum.

Soru 6: Trigger (Tetikleyici) nedir? Sen nerede kullandın?

Cevap: Trigger, bir tabloda `INSERT`, `UPDATE` veya `DELETE` işlemi olduğunda **kullanıcıdan bağımsız olarak otomatik tetiklenen** SQL kodlarıdır. Ben projede 3 adet trigger kullandım (örneğin `trg_Doktor_Log` ve `trg_Randevu_Log`). Bir doktorun bilgisi güncellendiğinde veya silindiğinde, trigger devreye girip değişikliği yapar ve işlemi alıp `Table_kullanici_log` tablosuna sessizce kaydeder.

Soru 7: Function (Fonksiyon) ile Stored Procedure arasındaki fark nedir?

Cevap: * **Stored Procedure**, tablolara veri ekleyebilir, güncelleyebilir veya silebilir (DML işlemleri). İçinde Transaction kullanılabilir. * **Function** ise genellikle DML (Insert/Update) işlemi yapmaz, amacı parametre alıp hesaplama yaparak tekil bir değer (veya tablo) döndürmektir (Return kullanır). *Projeden örnek:* `fn_HastaRandevuSayisi` adında bir fonksiyon yazdım. Bu fonksiyona hastanın TC'sini verdiğinizde hastanın toplam randevu sayısını INT olarak döndürüyor. Bu fonksiyonu View'ların içerisinde anlık hesaplama yapmak için kullandım.

Soru 8: Silme (Delete) işlemlerini nasıl yaptın?

Cevap: Doktor ve Randevu silme işlemlerini kalıcı olarak (`DELETE FROM`) sildim. Fakat Hasta gibi ilişkisi çok olan tablolarda **Soft-Delete (Yazılımsal Silme)** mantığı kullandım. `sp_HastaSil` prosedürü hastayı tablodan fiziksel olarak silmiyor, onun yerine `aktif = 0` yapıyor. Böylece hastanın eski reçete veya randevu geçmişi veritabanında "Foreign Key (Yabancı Anahtar)" hatası vermeden saklanmaya devam ediyor, sadece sistemde listelenmiyor.

Sunum Taktikleri (Ekstra Notlar): * Hocaya gösterirken *"Hocam bu ekranı C#'ta hazırladım ama istatistikleri ve hastaları çekerken View kullandım"* deyin. * C# uygulamasında rastgele bir doktorun adını değiştirip **Kaydet** deyin. Sonra **Sistem Logları** butonuna tıklayın. *"Bakın hocam, benim kodum log tablosuna müdahale etmiyor, veritabanındaki Trigger değişikliği algılayıp buraya kendi yazdı"* şeklinde gösterin. Notunuzu ciddi oranda artıracaktır!